When the guardian receives `Start`, it passes the list of texts to the manager, which splits the list and assigns one piece per worker. The application produces the following output:

```
INFO  [Guardian$] - setting up. Creating manager
INFO  [Manager$] - sending text '-one-' to worker
INFO  [Manager$] - sending text '--two--' to worker
INFO  [Worker$] - 'Actor[akka://error-kernel/user/manager-alpha/
➥worker-one-#1892512941]' DONE!. Parsed result: one
INFO  [Worker$] - 'Actor[akka://error-kernel/user/manager-alpha/
➥worker--two--#1275513111]' DONE!. Parsed result: two
INFO  [Manager$] - text '-one-' has been finished

INFO  [Manager$] - text '--two--' has been finished
```

Let’s focus on the first three lines of this output: how the guardian creates the manager and the manager delegates tasks during setup. To do this, you need two new tools from the Akka toolkit:

- `Spawn`—To create actors from another actor, you use the context by using `context.spawn([someActorsApplyMethod], [someName])`. The signature is the same as that for the `ActorSystem` and creates an `ActorRef` to which you can send messages. In the next example, the spawning is done inside a `Behaviors.setup`.
- `Behaviors.setup`—This factory creates a behavior that is executed only once: when the actor is instantiated. It creates a behavior from a function with only one input parameter: the `context: ActorContext` used to spawn the manager.

Once the guardian is set up, it uses the manager to delegate the parsing of the texts in the `Start` message.